AI Engineer Self-Learning System - Checkpoint Guide

This guide provides 5 hands-on checkpoints to get your self-learning AI Engineer system up and running. Each checkpoint is designed to take less than 20 minutes and provides visible wins.

Prerequisites

- Docker and Docker Compose installed
- Python 3.8+ with pip
- Basic familiarity with terminal/command line
- Optional: Slack workspace for notifications

Checkpoint 1: Launch the MCP Learning Server (15 minutes)

Goal: Get the core MCP Learning Server running and verify it's accepting requests.

Steps:

1. **Start the MCP Server**:
   ```bash
   cd quantum-loop-debugger
   python assistant/mcp_simple_server.py
   ```

2. **Verify Server is Running**:
   Open a new terminal and test the endpoints:
   ```bash
   # Check server status
   curl http://localhost:8084/status

# Test memory storage
curl -X POST http://localhost:8084/memory \
-H "Content-Type: application/json" \
-d '{"type": "test", "content": "First memory entry", "importance": 3}'

# Retrieve stored memory
curl http://localhost:8084/memory?limit=5
```

1. **Test Feedback Endpoint**:
   ```bash
   curl -X POST http://localhost:8084/feedback \
     -H "Content-Type: application/json" \
     -d '{"action": "test_action", "result": "success", "success": true, "duration": 1.5}'
   ```

Success Indicators:

- ✅ Server starts without errors on port 8084
- ✅ Status endpoint returns server information

- ✅ Memory and feedback can be stored and retrieved
- ✅ JSON responses are properly formatted

Troubleshooting:

- **Port already in use**: Change port in `mcp_simple_server.py` line 200
- **Permission errors**: Ensure `/workspace` directory is writable
- **Import errors**: Install missing dependencies with `pip install flask requests`

Checkpoint 2: Launch the Simple Dashboard (15 minutes)

Goal: Get the Tkinter dashboard running and see real-time metrics from the MCP server.

Steps:

1. **Install GUI Dependencies** (if needed):
   ```bash
   # On Ubuntu/Debian
   sudo apt-get install python3-tk

# On macOS (with Homebrew)
brew install python-tk
```

1. **Start the Dashboard**:
   `bash`
   `python tk_simple_dashboard.py`

2. **Interact with Dashboard**:
   - Observe the “System Status” panel showing server metrics
   - Click “Rotate Model” to see model switching
   - Watch the “Recent Activity” log for updates
   - Check “Memory Updates Visualization” for stored entries

3. **Generate Some Activity**:
   In another terminal, create some test data:
   ```bash
   # Add more memory entries
   for i in {1..3}; do
   curl -X POST http://localhost:8084/memory \
   -H “Content-Type: application/json” \
   -d “{"type": "checkpoint2", "content": "Test entry $i", "importance": $i}”
   done

# Add feedback entries
curl -X POST http://localhost:8084/feedback \
-H “Content-Type: application/json” \
-d ‘{“action”: “dashboard_test”, “result”: “completed”, “success”: true, “duration”: 2.1}’
```

Success Indicators:

- ✅ Dashboard window opens and displays properly
- ✅ “System Status” shows green “Healthy” status
- ✅ Memory and feedback counts update automatically
- ✅ Activity log shows real-time updates
- ✅ Model rotation works when button is clicked

Troubleshooting:

- **GUI doesn’t open**: Ensure X11 forwarding is enabled (if using SSH)
- **Connection errors**: Verify MCP server is running on localhost:8084
- **Update issues**: Check firewall settings for local connections

---

Checkpoint 3: Test Knowledge Ingestion (20 minutes)

Goal: Process a PDF document and see insights extracted and stored in the MCP server.

Steps:

1. **Create a Test PDF** (or use an existing one):
   ```bash
   # Create a simple test document
   echo “This is an important finding about artificial intelligence.
   The key insight is that machine learning models can improve over time.
   We recommend implementing continuous learning systems.
   The results show significant improvements in accuracy.” > test_document.txt

# Convert to PDF (if you have pandoc installed)
pandoc test_document.txt -o test_document.pdf
```

1. **Install PDF Processing Dependencies**:
   `bash`
   `pip install PyPDF2 requests`

2. **Test Knowledge Processing**:
   ```python
   # Run this in Python interactive shell or create a test script
   from assistant.knowledge_simple import SimpleKnowledgeProcessor

processor = SimpleKnowledgeProcessor()
result = processor.process_pdf(“test_document.pdf”)
print(json.dumps(result, indent=2))
```

1. **Verify Integration**:
   `bash`
   `# Check that insights were stored in MCP server`
   `curl http://localhost:8084/memory?type=knowledge_insight&limit=10`

2. **Check Dashboard**:
   - Look at the dashboard to see new memory entries

- Verify the memory count has increased
- Check the memory visualization section

Success Indicators:

- ✅ PDF text is successfully extracted
- ✅ 3 key insights are identified and extracted
- ✅ Insights are stored locally in `/workspace/knowledge/`
- ✅ Insights are sent to MCP server as memory entries
- ✅ Dashboard shows increased memory count
- ✅ Memory visualization displays the new insights

Troubleshooting:

- **PDF reading errors**: Ensure PDF is not password-protected or corrupted
- **No insights extracted**: Try with a longer, more content-rich document
- **MCP connection fails**: Verify server is running and accessible

---

Checkpoint 4: Docker Deployment (18 minutes)

Goal: Deploy the entire system using Docker Compose and verify all services work together.

Steps:

1. **Stop Running Services**:
   Stop any manually running MCP server or dashboard instances.

2. **Build and Start with Docker**:
   ```bash
   # Build the containers
   docker-compose build

# Start all services
docker-compose up -d

# Check service status
docker-compose ps
```

1. **Verify Services**:
   ```bash
   # Check MCP Learning Server
   curl http://localhost:8084/status

# Check main AI Engineer service
curl http://localhost:8080/health

# Check enhanced dashboard
curl http://localhost:5000
```

1. **Test Integration**:
   ```bash
   ```

```
# Add test data through Docker services
curl -X POST http://localhost:8084/memory \
-H "Content-Type: application/json" \
-d '{"type": "docker_test", "content": "Docker deployment successful", "importance": 4}'

# Check logs
docker-compose logs mcp-learning
```

1. **Access Web Dashboard**:
   Open browser to `http://localhost:5000` to see the enhanced web dashboard.

Success Indicators:

- ✅ All containers start successfully
- ✅ No error messages in `docker-compose ps`
- ✅ MCP server responds on port 8084
- ✅ Web dashboard accessible on port 5000
- ✅ Services can communicate with each other
- ✅ Persistent volumes are working (data survives container restart)

Troubleshooting:

- **Build failures**: Check Dockerfile syntax and dependencies
- **Port conflicts**: Modify ports in docker-compose.yml if needed
- **Volume issues**: Ensure Docker has permission to create volumes
- **Service communication**: Check network configuration in docker-compose.yml

---

Checkpoint 5: n8n Workflow Integration (17 minutes)

Goal: Set up n8n workflow to capture MCP events and send Slack notifications.

Steps:

1. **Start n8n** (if not already running):
   ```bash
   # Using Docker
   docker run -it –rm –name n8n -p 5678:5678 n8nio/n8n

# Or using npm
npx n8n
```

1. **Import Workflow**:
   - Open n8n at `http://localhost:5678`
   - Go to "Workflows" → "Import from File"
   - Upload the `n8n_workflow.json` file
   - Activate the workflow

2. **Configure Credentials** (Optional - for Slack):
   - Go to "Credentials" in n8n
   - Add Slack API credentials if you want real notifications
   - For testing, you can skip this and use the webhook capture only

3. **Test Webhook**:
   ```bash
   # Get the webhook URL from n8n (usually something like):
   # http://localhost:5678/webhook/ai-engineer-events

# Send test event
curl -X POST http://localhost:5678/webhook/ai-engineer-events \
-H "Content-Type: application/json" \
-d '{
"event_type": "memory_update",
"message": "New memory entry added",
"timestamp": "'$(date -u +%Y-%m-%dT%H:%M:%SZ)'",
"source": "MCP Server",
"success_rate": 85.5,
"memory_count": 15,
"feedback_count": 8
}'
```

1. **Verify Workflow Execution**:
   - Check n8n execution history
   - Verify webhook response
   - If Slack is configured, check for notification

2. **Integrate with MCP Server** (Advanced):
   Modify the MCP server to send webhooks on events:
   ```python
   # Add this to mcp_simple_server.py in the update_status function
   import requests

def send_webhook_notification(event_type, message):
try:
webhook_url = "http://localhost:5678/webhook/ai-engineer-events"
data = {
"event_type": event_type,
"message": message,
"timestamp": datetime.now().isoformat(),
"source": "MCP Server",
"success_rate": server_status.get("success_rate", 0),
"memory_count": server_status.get("memory_entries", 0),
"feedback_count": server_status.get("feedback_entries", 0)
}
requests.post(webhook_url, json=data, timeout=5)
except:
pass # Fail silently to not break main functionality
```

Success Indicators:

- ✅ n8n workflow imports successfully
- ✅ Webhook endpoint is accessible
- ✅ Test webhook triggers workflow execution

- ✅ Workflow completes without errors
- ✅ If Slack configured: notification appears in channel
- ✅ Database logging works (if PostgreSQL configured)

Troubleshooting:

- **n8n won't start**: Check if port 5678 is available
- **Workflow import fails**: Verify JSON syntax in workflow file
- **Webhook not triggering**: Check URL and HTTP method
- **Slack errors**: Verify API credentials and channel permissions

---

System Verification Checklist

After completing all checkpoints, verify your complete system:

Core Functionality:

- [ ] MCP Learning Server running and responding
- [ ] Dashboard showing real-time metrics
- [ ] Knowledge ingestion processing PDFs
- [ ] Docker deployment working
- [ ] n8n workflow capturing events

Data Flow:

- [ ] Memory entries stored and retrieved
- [ ] Feedback tracking success/failure rates
- [ ] Knowledge insights integrated with memory
- [ ] Dashboard updates automatically
- [ ] Webhook notifications working

Persistence:

- [ ] Data survives service restarts
- [ ] Docker volumes maintain state
- [ ] Configuration persists

Next Steps

Once all checkpoints are complete, you have a fully functional self-learning AI Engineer system! Consider these enhancements:

1. **Add More Knowledge Sources**: Integrate with APIs, databases, or other document types
2. **Enhance Learning**: Implement more sophisticated insight extraction
3. **Expand Notifications**: Add email, Teams, or other notification channels
4. **Add Authentication**: Secure your endpoints with API keys or OAuth
5. **Scale Up**: Deploy to cloud infrastructure for production use

Support

If you encounter issues:

1. Check the troubleshooting sections for each checkpoint
2. Verify all prerequisites are installed
3. Check service logs: `docker-compose logs [service-name]`
4. Ensure all ports are available and not blocked by firewall
5. Test each component individually before integration

Remember: Each checkpoint should provide visible progress and working functionality. Don’t move to the next checkpoint until the current one is fully working!